

GenAI Application Development: Phase 1

Study Guide: Foundation & Model Selection on Databricks

1. Select a model from a model hub or marketplace

Source: <https://docs.databricks.com/aws/en/marketplace/> & <https://docs.databricks.com/aws/en/marketplace/get-started-consumer>

Core Concept: Databricks Marketplace acts as a secure, open exchange where data consumers can discover and immediately access datasets, machine learning models, and Model Context Protocol (MCP) servers without duplicating data.

How it works on Databricks: The Marketplace uses OpenSharing and is deeply integrated with Unity Catalog (UC). Datasets and AI models are shared in read-only catalogs directly in your workspace. You can install pre-packaged models or external MCP servers (which provide APIs for AI agents to interact with external systems) directly from the Marketplace UI.

When to use vs. alternatives:

- **Use Databricks Marketplace:** When you want immediate access to governed, secure assets via Unity Catalog, avoiding the overhead of manual data movement or third-party authentication logic.
- **Alternatives:** Downloading models directly from Hugging Face or using standard API calls. Use external integrations when a desired cutting-edge model isn't yet available on the Marketplace.

Common Mistakes: Failing to ensure the workspace is Unity Catalog-enabled, or lacking the `USE MARKETPLACE ASSETS` privilege on the metastore, preventing access to shared models.

2. Select the best LLM based on application attributes

Source: <https://docs.databricks.com/aws/en/machine-learning/foundation-model-apis/> & <https://docs.databricks.com/gcp/en/machine-learning/foundation-model-apis/priority-mode>

Core Concept: Mapping your application's requirements (latency, throughput, cost) to the correct Databricks Model Serving deployment tier.

How it works on Databricks: Foundation Model APIs offer multiple tiers:

- **Standard Pay-per-token:** Best-effort serving. Billed per token.
- **Priority Pay-per-token:** Opt-in for latency-sensitive apps. Admits requests ahead of standard traffic for better availability without capacity commitments. Accessed by setting the `service_tier` parameter to "priority".

- **Provisioned Throughput:** Dedicated endpoints with guaranteed capacity. Supports custom weights or fine-tuned base models.

When to use vs. alternatives: Use Standard for prototyping or batch workloads. Use Priority mode for real-time applications needing consistent performance under load. Use Provisioned Throughput for mission-critical production with high sustained throughput, strict latency Service Level Agreements (SLAs), or when serving a fine-tuned model variant.

Common Mistakes: Using standard pay-per-token for high-scale production, resulting in `429 Too Many Requests` errors when Input Tokens Per Minute (ITPM) or Output Tokens Per Minute (OTPM) limits are exceeded.

Code Example & Function: Querying a pay-per-token model using the OpenAI client.

```
import os
from openai import OpenAI

# Initialize client pointing to Databricks Workspace
client = OpenAI(
    api_key=os.environ.get("DATABRICKS_TOKEN"),
    base_url="https://serving-endpoints"
)

# Function: client.chat.completions.create()
# Sends inference requests to the specified Databricks endpoint
response = client.chat.completions.create(
    model="databricks-meta-llama-3-1-70b-instruct",
    messages=[{"role": "user", "content": "Explain Generative AI."}],
    max_tokens=256
)
print(response.choices[0].message.content)
```

3. Select the best model via experiment metrics

Source: <https://docs.databricks.com/aws/en/mlflow3/genai/eval-monitor/concepts/eval-harness>

Core Concept: Utilizing MLflow GenAI evaluation tools to systematically benchmark LLMs against test datasets using quantitative and qualitative "scorers."

How it works on Databricks: The `mlflow.genai.evaluate()` function runs your GenAI app against an evaluation dataset. It captures traces natively (via MLflow Tracing) and applies specific scorers, such as built-in LLM judges (e.g., Relevance to Query, Safety) or third-party metrics like DeepEval. Results and rationale are logged in an MLflow Experiment run for version comparison.

When to use vs. alternatives: Use MLflow Evaluation directly during development ("Direct Evaluation") rather than relying on manual inspection or unstructured answer sheets. Direct Evaluation lets MLflow invoke your endpoint automatically, generate traces, and score them systematically in one pass.

Common Mistakes: Evaluating third-party or rate-limited foundation models without wrapping the prediction function in rate-limiting logic (e.g., using the `ratelimit` Python library), which causes concurrent evaluation runs to fail with HTTP 429 errors.

Code Example & Function: Evaluating an LLM endpoint with built-in MLflow scorers.

```
import mlflow
import pandas as pd
from mlflow.genai.scorers import RelevanceToQuery, Safety

# Define evaluation dataset
eval_data = pd.DataFrame({
    "inputs": ["What is MLflow?", "How do I securely share data?"],
    "ground_truth": ["MLflow is an AI engineering platform.", "Use OpenSharing."]
})

# Function: mlflow.genai.evaluate()
# Runs the provided dataset against the specified model endpoint (or function)
# and calculates metrics using the defined scorers.
results = mlflow.genai.evaluate(
    model="endpoints:/my-chat-endpoint",
    data=eval_data,
    targets="ground_truth",
    scorers=[RelevanceToQuery(), Safety()]
)

print(results.metrics_summary)
```